

Role 2: Data Engineering Specialist

Project Name:	Automated Airfare Index Pipeline	Hackathon Scope:	Multi-Route (T+1 to T+45)
Submission Deadline:	Sept 3, 2026	Status:	VERIFIED & LOCKED
Tech Stack:	Python 3.13, Pydantic v2, Pandas, PyArrow, Parquet, FPDF2		

1. Executive Summary & Data Engineering Architecture

Role 2 transforms the raw audit envelopes captured by Role 1 into a trusted, columnar dataset ready for fare indexation and downstream analytics. The pipeline enforces a strict four-stage contract: schema-bound validation, regex-driven fare unbundling, statistical outlier filtering and a retrievable Parquet/JSONL hand-off. Every envelope is validated against a Pydantic v2 contract; malformed payloads are quarantined with machine-readable error reasons instead of being silently dropped.

Architecture at a glance:

- * `schema_validator.py` - ingests staging JSONL and validates each record against the Pydantic `RawStagingPayload` contract, separating valid records from quarantined ones.
- * `fare_unbundler.py` - uses regular expressions to decompose raw fare text into base fare, tax and airport UDF, and convenience/management fee components.
- * `outlier_filter.py` - applies the IQR rule partitioned by `route_code` and `advance_window`, with a Z-score fallback for small sample cohorts.
- * `clean_staging_lead.py` - orchestrates all stages sequentially and materializes the clean airfare index in Snappy-compressed Parquet and mirror JSONL.

2. Component Deep-Dive

2.1 Schema Validator & Quarantine Engine (`schema_validator.py` & `schemas.py`)

The engine reads `seed_data/staging_raw_payloads.jsonl` line by line and normalizes each envelope through the Pydantic v2 `RawStagingPayload` model. Required date strings (YYYY-MM-DD) and IATA airport codes (3 uppercase letters) are enforced at the boundary. Two failure classes are captured independently: malformed JSON text and schema-rule violations. Both are written to `seed_data/quarantine_raw_payloads.jsonl` with the source line number and a structured error reason, enabling full provenance and replay. Successful records are returned as validated model instances for downstream processing.

2.2 Regex Tax & Fare Unbundling Module (`fare_unbundler.py`)

The unbundler accepts raw fare text or HTML snippets and strips markup before extraction. Case-insensitive regular expressions target three currency-carrying fields: base fare (Base, Base Fare), tax and airport user development fee (Tax, Tax & Airport UDF), and convenience or management fee. Delimiters, currency symbols and thousands separators are handled defensively. If an explicit tax token cannot be matched but a total quote is available, a proportional 82/18 fallback reconstructs the base fare and tax components. The outcome is always a validated `UnbundledFare` model instance.

2.3 IQR Outlier & Anomaly Detection Layer (`outlier_filter.py`)

Fares are partitioned into homogeneous cohorts keyed by `route_code` and `advance_window`. For cohorts of ten or more observations the classical IQR fence rule is applied: lower bound = $Q1 - (1.5 \times IQR)$ and upper bound = $Q3 + (1.5 \times IQR)$. Sparse cohorts fall back to a population Z-score rule with a 3.0 standard deviation threshold. Flagged observations never leave the dataset; each row receives an `is_outlier` boolean and a descriptive `outlier_reason` string, keeping the decision surface fully auditable.

3. Data Transformation Schema & Parquet Hand-off Contract

The cleaning pipeline spans three strict contracts. (1) RawStagingPayload preserves the Role 1 audit envelope with timestamped interception metadata. (2) UnbundledFare decomposes the quote into non-negative base_fare, tax_udf, convenience_fee and total_quote components. (3) CleanAirfareRecord is the fully transformed output row whose model validator guarantees $\text{total_quote} \geq \text{base_fare}$. The materialized hand-off artifact is seed_data/clean_airfare_index.parquet compressed with the Snappy codec, written via PyArrow from the orchestrated pandas frame.

Output Parquet schema (12 typed columns):

route_code:	string		
origin:	string	departure_date:	string
destination:	string	advance_window:	string
captured_at:	string	base_fare:	float64
tax_udf:	float64	convenience_fee:	float64
total_quote:	float64	is_outlier:	bool
outlier_reason:	string		

Sample transformed JSONL record:

```
{"route_code": "DEL-BOM", "origin": "DEL", "destination": "BOM",  
  "advance_window": "T+1", "departure_date": "2026-08-28",  
  "base_fare": 4200.0, "tax_udf": 920.0, "convenience_fee": 300.0,  
  "total_quote": 5420.0, "is_outlier": false, "outlier_reason": ""}
```

4. Verification Protocol & Execution Audit Results

The Role 2 pipeline was executed locally and verified end to end.

Execution commands: `python -m src.engineering.clean_staging_lead` | `python -m src.engineering.verify_parquet` | `python -m pytest tests/test_data_engineering.py`

Audit Results:

1. Ingestion count: 30 envelopes read | 30 passed | 0 quarantined.
2. Validation pass rate: 100.00% across all staging records.
3. Unbundled fare accuracy: 30/30 fares decomposed with 100% component recovery.
4. Parquet compression ratio: staging JSONL 40,303 bytes -> Parquet 8,041 bytes (5.0x smaller, ~80% reduction).
5. Outlier detection: 0 flagged across the seed window (expected for sparse cohorts).
6. Critical null check: total_quote, base_fare and route_code all confirmed null-free.
7. Unit test suite: 22 tests collected and passing (pytest).

Conclusion: Role 2 is fully functional, schema-locked and verified for the September 3 internal hackathon submission.